ANO
2026

UNINTER

PROJETO MULTIDISCIPLINAR

TRILHA: BACK-END

REDE “RAIZES DO NORDESTE”

Prof. Me. Luciane Yanase Kanashiro

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

OBJETIVO

Olá a todos.

Sejam todos muito bem-vindos!

Esta Atividade Prática tem como objetivo avaliar a capacidade do estudante de analisar um estudo de caso, modelar dados, definir contratos de API e implementar uma solução de Back-end que atenda aos principais requisitos funcionais e não funcionais do cenário proposto.

O foco da trilha de Back-end é demonstrar domínio sobre:

- Levantamento e priorização de requisitos (o que é essencial no sistema e por que);
- Modelagem do domínio e da base de dados (DER e entidades, relacionamentos e integridade);
- Arquitetura e organização do projeto (camadas, separação de responsabilidades, padrões de projeto quando aplicável);
- Implementação de API REST com endpoints coerentes, contratos bem definidos e documentação via Swagger/OpenAPI;
- Persistência em banco de dados com consistência entre modelo e implementação (migrations/ORM, seed quando necessário);
- Regras de negócio do fluxo crítico (por exemplo: pedidos, estoque por unidade, atualização de status, fidelização e validações);
- Integração simulada de pagamento (mock) para demonstrar o fluxo completo sem depender de provedores reais;
- Segurança e LGPD (mínimo técnico): autenticação, autorização por perfis/roles, cuidado com dados pessoais, armazenamento seguro (ex.: hash de senha) e respostas sem exposição indevida de dados;
- Qualidade e testabilidade: plano de testes com cenários positivos e negativos e evidência reproduzível (coleção Postman/Insomnia e/ou testes automatizados).

Ao final, espera-se que o estudante seja capaz de apresentar uma solução que execute, seja reproduzível (README claro), e comprove o domínio dos elementos essenciais de um Back-end profissional, conforme os requisitos e limitações definidos neste roteiro.

No mais, desejo-lhe excelente atividade prática em nome dos professores da disciplina de Projeto Multidisciplinar.

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

SUMÁRIO

OBJETIVO ____ *1*

ORIENTAÇÕES GERAIS ____ *3*

INTRODUÇÃO E OBJETIVOS (OBRIGATÓRIO) ____ **3**

ANÁLISE E REQUISITOS (OBRIGATÓRIOS) ____ **4**

DIAGRAMAS (OBRIGATÓRIO) ____ **5**

API E ENDPOINTS (OBRIGATÓRIO) ____ **6**

LGPD, PRIVACIDADE E SEGURANÇA NO BACK-END(OBRIGATÓRIO) ____ **8**

ENTREGA TÉCNICA (OBRIGATÓRIA) ____ **8**

PLANO DE TESTES (OBRIGATÓRIO) ____ **10**

CRITÉRIOS DE ENTREGA E NORMAS DA ABNT (OBRIGATÓRIO) ____ **12**

CONCLUSÃO (OBRIGATÓRIO) ____ **12**

ORIENTAÇÃO FINAL ____ **12**

CRITÉRIOS DE AVALIAÇÃO ____ **13**

Critérios de Avaliação (Back-end) - Tabela Consolidada ____ **Erro! Indicador não definido.**

ANEXOS ____ **18**

EXEMPLO DE CASOS DE USO ____ 18

Modelo (template) para documentar endpoints ____ 20

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

ORIENTAÇÕES GERAIS

INTRODUÇÃO E OBJETIVOS (OBRIGATÓRIO)

Esta atividade simula um cenário real de mercado onde o aluno deve projetar uma solução robusta para uma rede de lanchonetes em expansão. O foco está na compreensão do negócio, na tomada de decisão técnica e na entrega de uma API/solução de Back-end que suporte múltiplos canais (App, Totem e Web).

CUIDADO!

Em programação, não existem dois códigos exatamente iguais. Cada programador organiza seu código de uma forma diferente, declara variáveis com nomes diferentes, faz comentários diferentes, gera mensagens aos usuários distintas etc. Por este motivo, não serão aceitos dois algoritmos idênticos entre alunos (ou iguais à Internet). Caso o corretor observe respostas iguais, elas serão consideradas como PLÁGIO e será atribuída a NOTA ZERO na questão.

TRECHO INSTITUCIONAL OMITIDO
NESTA COPIA EDITADA PARA ESTUDO

Para consultar as orientacoes completas,
utilize o documento oficial sem alteracoes.

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

ANÁLISE E REQUISITOS (OBRIGATÓRIOS)

O aluno deve listar os requisitos com foco em regras de negócio, persistência, integrações e segurança (não em telas).

- **Requisitos Funcionais (RF):** cadastro e autenticação (login), gestão de unidades da rede, cardápio por unidade, gestão de pedidos (criar, atualizar status, cancelar), controle de estoque, programa de fidelização e integração com serviço externo de pagamento (mock).
    - O projeto deve contemplar, **todos** os requisitos necessários ao completo funcionamento do sistema **incluindo,** ao menos, de forma conceitual, os **requisitos obrigatórios abaixo**:
        - Cadastro e autenticação de usuários (incluindo perfis/roles).
        - Visualização/consulta de cardápio por unidade (API).
        - Realização de pedidos (com itens, valores, status).
        - Atualização do status do pedido (cozinha → pronto → entregue / cancelado).
        - Controle de estoque (entrada/saída e restrição de venda por indisponibilidade).
        - Programa de fidelização (pontos e resgate simples, com consentimento).
        - Promoções/campanhas (ao menos como regra/documentação de como aplicar).
        - Solicitação de pagamento via serviço externo (mock) + registro. **Você não deve implementar pagamento real**, mas deve representar o fluxo do envio do pagamento e o retorno da API (status e payload) para o cliente (App/Totem/Web).
    - **Multicanalidade (domínio) (Obrigatório)**

      Conforme o estudo de caso, a solução deve atender múltiplos canais (ex.: APP, TOTEM, BALCÃO, PICKUP, WEB). Assim, o back-end deve tratar o canal de origem do pedido como um dado de domínio, registrando-o no pedido e garantindo rastreabilidade operacional entre canais.

      Requisito mínimo:

        - O pedido deve possuir um campo `canalPedido` (ENUM), com valores como: `APP`, `TOTEM`, `BALCAO`, `PICKUP`, `WEB`.
        - A criação de pedido deve exigir o preenchimento de `canalPedido`.
        - A API deve permitir consultar/filtrar pedidos por canal (ex.: query param `?canalPedido=TOTEM`).

      Objetivo: permitir consolidação e acompanhamento do fluxo de pedidos por canal, mantendo integração e consistência do atendimento na rede.

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

- **Requisitos Não Funcionais (RNF):** segurança (LGPD, controle de acesso, senha com hash, token), logs/auditoria de ações sensíveis, desempenho em horários de pico, disponibilidade do sistema, tolerância a falhas na integração de pagamento e documentação (OpenAPI/Swagger).

DIAGRAMAS (OBRIGATÓRIO)

Esta seção fundamenta a lógica da solução e é requisito essencial para a avaliação da visão sistêmica.

4.1. Diagrama de Casos de Uso

Elaboração de Diagrama de Casos de Uso com os atores:

- Cliente (App/Web/Totem);
- Atendente (Balcão);
- Cozinha;
- Gerente / Administrador;
- Sistemas Externos de Pagamento (Gateway).

Descrição de Feature (obrigatória): escolha as funcionalidades críticas (ex.: Realizar Pedido + Solicitar Pagamento) e detalhe fluxo principal, pré-condições, pós-condições, exceções e regras de negócio (ex.: idempotência, estoque insuficiente, pagamento negado).

4.2. DER (Diagrama Entidade-Relacionamento) / Modelo de Dados

Apresente o DER do banco de dados com entidades, atributos principais e relacionamentos. Inclua chaves (PK/FK), cardinalidades e restrições relevantes (ex.: uma unidade possui estoque próprio; pedido possui itens; pagamento é desacoplado).

4.3. Arquitetura (camadas e separação de responsabilidades) (obrigatório)

Para fins de organização e manutenção do projeto, a solução deve apresentar uma estrutura por camadas (ou equivalente), deixando explícita a separação de responsabilidades entre:

- **Domain (Domínio)**: entidades, regras de negócio, validações e comportamentos do domínio (ex.: Pedido, Cliente, Produto, Estoque).
- **Application (Aplicação)**: casos de uso/serviços que orquestram o fluxo do sistema (ex.: criar pedido, aplicar fidelidade, confirmar pagamento mock, atualizar status).
- **Infrastructure (Infraestrutura)**: persistência (ORM/migrations/repositórios), integrações (pagamento mock, e-mail/log), e detalhes técnicos.
- **API (Interface/Controllers)**: rotas/endpoints, autenticação/autorização, contratos de request/response e documentação (Swagger/OpenAPI).

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

Observação importante: você pode adotar variações como MVC, Clean Architecture ou DDD simplificado, desde que a separação de responsabilidades fique clara no código e na documentação. A avaliação considera a coerência e clareza da organização, e não a adesão a um framework específico.

4.4. Diagrama de Classes (obrigatório) e Sequência/Atividade (recomendado)

Apresente um diagrama de classes (visão de domínio) com as principais entidades/objetos e relacionamentos. Recomenda-se também um diagrama de sequência ou atividade do fluxo crítico (Pedido → Pagamento Externo → Atualização de Status).

API E ENDPOINTS (OBRIGATÓRIO)

Descreva os principais endpoints da sua API, organizados por recurso (ex.: /auth, /unidades, /produtos, /estoque, /pedidos, /fidelidade, /pagamentos). Para cada endpoint, apresente: método HTTP, rota, autenticação/permissões, parâmetros, exemplo de request/response, códigos de status e padrão de erro.

5.1 Objetivo desta etapa

Deve-se descrever o contrato da sua API (o “combinado” entre Front-end e Back-end), para que qualquer pessoa consiga:

1. entender **o que cada endpoint faz**,
2. saber **o que enviar** (request) e **o que receber** (response),
3. validar **regras e erros**,
4. testar com Postman/Insomnia/Swagger.

Importante: prefira qualidade e clareza. É melhor ter os endpoints bem documentados do que endpoints sem padrão.

5.2 Organização por recurso

Liste os endpoints agrupando por “recurso” (módulo), **por exemplo**:

- /auth (login, refresh, logout)
- /usuarios (cadastro, perfil)
- /unidades (listar unidades)
- /produtos (CRUD, consulta)
- /estoque (entradas/saídas, consulta por unidade)

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

- /pedidos (criar pedido, status, consulta)
- /pagamentos (simulação/mock, confirmação)
- /fidelidade (pontos, saldo, histórico)

5.3 O que você precisa apresentar para CADA endpoint (checklist)

Para cada endpoint, documente:

1. Nome do endpoint (propósito em 1 frase)
2. Método HTTP + rota
3. Autenticação e permissões (ex.: JWT; papel ADMIN/GERENTE/CLIENTE)
4. Parâmetros
    - Path params (ex.: `/produtos/{id}`)
    - Query params (ex.: `?page=1&limit=10`)
5. Body (request) com exemplo JSON
6. Response (sucesso) com exemplo JSON
7. Códigos de status esperados (ex.: 200, 201, 400, 401, 403, 404, 409, 422)
8. Padrão de erro (um JSON padrão para todas as falhas)

5.4 Regras mínimas (para evitar "API bagunçada")

- **URLs no plural:** `/produtos, /pedidos`
- **IDs no path:** /produtos/{id}
- **Paginação em listagens:** `GET /produtos?page=1&limit=10`
- **Status code coerente:**
    - 200 (ok), 201 (criado), 204 (sem conteúdo)
    - 400/422 (erro de validação), 401 (não autenticado), 403 (sem permissão)
    - 404 (não encontrado), 409 (conflito/regra de negócio)
- **Erro padronizado:** sempre o mesmo formato JSON

5.5 Campos e padrões mínimos do contrato (Obrigatório)

Nos contratos de pedido, inclua explicitamente o campo `canalPedido` e mantenha consistência de validação e erros. Caso `canalPedido` não seja informado ou seja inválido, a API deve retornar erro com status apropriado (ex.: 400/422) e mensagem padronizada.

Exemplo de request (Pedido — criação):

```
{
  "canalPedido": "TOTEM",
  "clienteId": 123,
```


VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

```
  "itens": [
    { "produtoId": 10, "quantidade": 2 }
  ],
  "formaPagamento": "MOCK"
}
```

Exemplo de filtro (listagem):

```
GET /pedidos?canalPedido=APP&status=AGUARDANDO_PAGAMENTO
```

LGPD, PRIVACIDADE E SEGURANÇA NO BACK-END (OBRIGATÓRIO)

A LGPD deve ser demonstrada explicitamente no Back-end: quais dados pessoais são coletados, para qual finalidade, por qual base legal e como o consentimento é registrado. Inclua controles mínimos: hashing de senha, autenticação por token, autorização por perfil, logs de acesso a dados sensíveis, e estratégia de retenção/anônimização quando aplicável.

ENTREGA TÉCNICA (OBRIGATÓRIA)

O aluno deve entregar uma API funcional (rodando localmente) + documentação técnica completa que permita a correção e teste do sistema.

7.1 Entregáveis obrigatórios:

1. **Repositório Git público** (GitHub/GitLab/Bitbucket) contendo **— obrigatório**:
   a) Código-fonte do Back-end
   b) Estrutura organizada (camadas, módulos, pastas)
   c) Histórico de commits (mínimo recomendado: 5 commits com evolução do trabalho)
2. **README de execução (passo a passo) — obrigatório:**
   a) Deve conter, no mínimo:
   b) Requisitos (linguagem, versão, banco, dependências)
   c) Como configurar variáveis de ambiente (.env.example + instruções)
   d) Como instalar dependências
   e) Como criar banco e executar migrations (e seed, se houver)
   f) Como iniciar a API
   g) Como acessar a documentação (Swagger/OpenAPI)
   h) Como rodar os testes (se houver)
3. **Documentação da API (OpenAPI/Swagger) — obrigatória**
   a) Pode ser gerada automaticamente pela ferramenta do projeto
   b) Deve refletir os endpoints implementados (contrato real)

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

c) Deve conter exemplos de request/response e códigos de status

4. **Coleção de testes Postman/Insomnia (arquivo .json) — obrigatória**
   a) Deve incluir chamadas do **fluxo principal** (mínimo viável) e evidências de:
      a. autenticação (login e uso do token)
      b. criação/consulta/atualização conforme o fluxo escolhido
      c. pelo menos 1 cenário de erro (ex.: 401, 404, 409 ou 422)
5. **Banco de dados e modelo (DER) — obrigatório**
   a) Entregar o DER (imagem/PDF) e indicar as principais tabelas e relacionamentos
   b) O DER deve ser compatível com o banco usado na API

7.2 Requisito mínimo de funcionamento (MVP obrigatório)

A API precisa rodar e demonstrar pelo menos um fluxo completo e testável, com persistência em banco:

Fluxo obrigatório (escolher 1 e entregar completo):

- **Fluxo A (recomendado): Pedido → Pagamento mock → Atualização de status**
  - criar pedido
  - validar itens / regras básicas
  - registrar pagamento mock
  - atualizar status do pedido
- **Fluxo B: Estoque por unidade**
  - cadastrar/consultar produto
  - movimentar estoque (entrada/saída)
  - consultar saldo por unidade

Obrigatório em qualquer fluxo

- Persistência em banco (CRUD real, não só mock em memória)
- Pelo menos **1 tipo de autenticação** (ex.: JWT) com **perfis/permissões** (mesmo que simples)
- Padrão de erro consistente (JSON padronizado)

Observação importante: Entregar “muitos endpoints” não compensa se o fluxo principal não estiver fechado e testável.

7.3 Evidências (obrigatório)

No README, incluir links e evidências:

- Link do repositório
- “URL do swagger local” + “rota do swagger” ou instrução para acessar o Swagger
- Arquivo da coleção Postman/Insomnia no repositório
- (Opcional) Link de deploy, se existir

AVISO CRÍTICO (NOTA ZERO)

Independentemente da tecnologia escolhida, o aluno deve garantir que os links estejam **PÚBLICOS** e **funcionais**. Se o avaliador não conseguir acessar **repositório/Swagger/coleção Postman** no momento da correção, **a pontuação do critério “Entrega Técnica e Documentação” será 0 (zero)**.

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

PLANO DE TESTES (OBRIGATÓRIO)

Definir e evidenciar como a API foi validada, garantindo que o corretor consiga reproduzir os testes usando a coleção Postman/Insomnia entregue no repositório e/ou o Swagger/OpenAPI.

8.1 Evidência obrigatória

Você deve entregar:

- Coleção Postman/Insomnia (.json) com os testes organizados em pastas (ex.: Auth, Produtos, Pedidos, Pagamento, Erros); e
- No README, indicar como executar os testes (ordem sugerida, token, ambiente, seed).

Observação: os cenários descritos abaixo devem estar refletidos na coleção (ou seja: não basta descrever, tem que executar).

8.2 Cenários mínimos (mínimo 10)

Apresentar no mínimo 10 cenários de teste, sendo:

- mínimo 6 positivos (fluxo esperado)
- mínimo 4 negativos (erros/regras)

Cada cenário deve conter:

- **ID do teste** (T01, T02…)
- **Endpoint + método**
- **Pré-condição** (ex.: usuário logado; produto existente; seed aplicado)
- **Entrada** (path/query/body)
- **Saída esperada**
  - **status code**
  - **trechos relevantes do response**
- **Evidência** (nome da requisição na coleção Postman/Insomnia e/ou print do Swagger com exemplo)

8.3 Os testes devem cobrir obrigatoriamente:

a) Autenticação e autorização

a. Login (token válido)
b. Acesso sem token (401)
c. Acesso com perfil sem permissão (403)

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

b) Validação de dados

a. Campo obrigatório ausente (400/422)
b. Tipo/formato inválido (ex.: e-mail, quantidade negativa) (400/422)

c) Regras de negócio do fluxo principal

Escolha o fluxo adotado no projeto e cubra ao menos:

a. Pedido com itens válidos → criado (ex.: 201)
b. Pedido com produto/unidade inexistente (404)
c. Pedido com estoque insuficiente (409 ou regra equivalente)

d) Pagamento mock e atualização de status

a. Pagamento mock aprovado → status do pedido atualizado
b. Pagamento mock recusado → status e mensagem coerentes

e) Logs/auditoria (quando implementado)

a. Pelo menos 1 teste evidenciando que uma ação sensível gera registro (ex.: criação de pedido, mudança de status)

Observação: Se você não implementou logs/auditoria: declarar explicitamente "não implementado" e justificar (isso impacta nota, mas evita inconsistência).

8.4 Formato recomendado

ID	Cenário	Endpoint	Pré-condição	Entrada	Esperado (status + pontos do response)	Evidência (nome na coleção)
T01	Login válido	POST /auth/login	usuário cadastrado	{email, senha}	200 + accessToken	Auth/Login válido
T02	Sem token	GET /pedidos	—	—	401 + erro padrão	Pedidos/Listar sem token
…	…	…	…	…	…	…

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

CRITÉRIOS DE ENTREGA E NORMAS DA ABNT (OBRIGATÓRIO)

O trabalho deve ser entregue em **arquivo único (PDF)** e seguir rigorosamente as **normas da ABNT** para trabalhos acadêmicos.

- **Onde encontrar as normas:** No menu lateral esquerdo do AVA, acesse **Biblioteca** e clique em **Normas ABNT**.
- O trabalho deve conter capa, sumário, seções do roteiro e referências.
- **Postura Profissional:** O projeto deve ser apresentado como uma solução real que se sustentaria em uma entrevista técnica de mercado.
- **Como nomear seu arquivo:** Utilize o seu RU seguido de Projeto Back End. Exemplo: **4999657_Projeto_Back_End.PDF**

CONCLUSÃO (OBRIGATÓRIO)

Resuma o que foi implementado e o que ficou como proposta, justificando prioridades. Explique como DER/Classes/Casos de Uso se conectam aos endpoints e ao fluxo crítico entregue. Descreva como o pagamento mock foi tratado e quais validações/erros foram padronizados. Indique os principais cuidados de segurança/LGPD e como os testes evidenciam o funcionamento.

ORIENTAÇÃO FINAL

Antes de finalizar e realizar a entrega do trabalho, reflita: **“Se este projeto fosse apresentado em uma entrevista técnica ou empresa, ele se sustentaria?"**

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

CRITÉRIOS DE AVALIAÇÃO

Transparência de avaliação:

Os critérios abaixo são apresentados em duas formas equivalentes (texto e tabela) para facilitar a compreensão. Em caso de divergência de interpretação, considere a **Tabela Consolidada** como referência principal, pois ela resume a distribuição de pontos e faixas de desempenho.

A nota será distribuída da seguinte forma:

- **20 PONTOS: Análise do Problema e Requisitos (20 pontos)**: Avalia o entendimento do estudo de caso e tradução do cenário em requisitos de Back-end.
    - **Serão considerados:**
        - compreensão do contexto (rede/franquia; matriz e unidades);
        - requisitos funcionais e não funcionais específicos de Back-end (API, dados, segurança, integrações);
        - menção objetiva a fidelização, pagamento externo (mock) e LGPD (mínimo);
        - priorização e justificativa técnica (o que foi implementado vs. planejado).
        - multicanalidade como requisito do domínio (registro do canalPedido e rastreabilidade por canal);
        - priorização/justificativas técnicas do MVP incluindo canal, pagamento mock e LGPD.
    - **Critérios de avaliação:**
        - Insuficiente (0–8): requisitos desconectados do estudo de caso, sem priorização.
        - Básico (9–12): requisitos corretos, mas genéricos e com pouca justificativa.
        - Bom (13–16): requisitos claros e coerentes, com priorização e justificativas.
        - Excelente (17–20): visão sistêmica clara, com justificativas técnicas e consideração explícita de multicanalidade, LGPD e integrações (mock).

- **20 PONTOS: Modelagem e Arquitetura de Back-end (20 pontos):** Avalia a capacidade de representar dados, atores e fluxos essenciais e sua coerência com o que foi implementado.
-
    - **Serão considerados:**
        - Diagrama de Casos de Uso;
        - DER/Modelo de dados;
        - Diagrama de Classes (domínio) e/ou Sequência/Atividade do fluxo crítico;
        - Coerência entre modelagem, regras de negócio e endpoints.

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

- organização em camadas (Domain / Application / Infrastructure / API) ou equivalente, com separação clara de responsabilidades;
- coerência entre modelagem (DER/diagramas), regras de negócio e endpoints implementados/documentados.

- **Critérios de avaliação:**
  - Insuficiente (0–3): artefatos ausentes/incompletos/incoerentes.
  - Básico (4–8): artefatos corretos, porém rasos ou pouco conectados.
  - Bom (9–14): modelagem consistente e conectada ao fluxo crítico.
  - Excelente (15–20): modelagem madura, clara, com justificativas e alinhamento total ao que foi entregue.

- **25 PONTOS**: **Implementação da API e Regras de Negócio (25 pontos):** Avalia a entrega técnica do Back-end (funcionamento real) e aderência aos requisitos.
  - Serão considerados:
    - API **executável** conforme README;
    - endpoints implementados e **documentados em Swagger/OpenAPI**;
    - persistência em banco (ORM/migrations) e consistência com o DER;
    - regras de negócio do fluxo principal (pedido/estoque/status e validações);
    - pagamento mock com **cenário de sucesso e falha**.
    - registro e uso do campo `canalPedido` no fluxo do pedido (criação e consultas), conforme requisito de multicanalidade;
    - aderência dos endpoints e regras de negócio ao fluxo crítico definido, com validações e padrão de erro.
  - **MVP mínimo esperado**:
    - autenticação (ex.: JWT) + pelo menos 1 regra de autorização (perfil/role);
    - 1 fluxo crítico fechado ponta a ponta (pedido→pagamento mock→status, ou estoque→movimentação→saldo por unidade).
  - **Critérios de avaliação:**
    - Insuficiente (0–5): API não executa / não fecha o fluxo mínimo / sem persistência real.
    - Básico (6–10): CRUD parcial, regras fracas ou documentação incompleta.
    - Bom (11–17): fluxo principal concluído, persistência ok, endpoints coerentes.
    - Excelente (18–25): solução madura (padrão de erro, validações, paginação quando aplicável, consistência de status codes e boas práticas).

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

- **15 PONTOS: Segurança, LGPD, Logs e Auditoria (15 pontos):** Avalia privacidade, controle de acesso e rastreabilidade **na prática**, não só “citação”.
    - Serão considerados:
        - autenticação e autorização (perfis/roles) **aplicadas em endpoints**;
        - LGPD mínimo: finalidade + minimização + cuidado com dados pessoais/sensíveis;
        - armazenamento seguro (hash de senha; não expor dados sensíveis em responses);
        - logs/auditoria: registro de ações sensíveis (ex.: mudança de status, criação/cancelamento de pedido).
    - **Critérios de avaliação:**
        - Insuficiente (0–3): ausente ou apenas citado sem aplicação.
        - Básico (4–7): controles mínimos, mas incompletos ou inconsistentes.
        - Bom (8–11): aplicação coerente (auth/roles + LGPD mínimo + logs básicos).
        - Excelente (12–15): abordagem integrada (rastreabilidade clara + justificativas + cuidados consistentes)

- **10 PONTOS: Plano de Testes (API) (10 pontos):** Avalia a capacidade de validar a API e evidenciar funcionamento de **forma reproduzível**.
    - Serão considerados:
        - mínimo 10 cenários (positivos e negativos) com entradas e saídas esperadas (status + resposta);
        - evidência preferencial: coleção Postman/Insomnia com os cenários organizados (prints podem complementar, mas não substituem a coleção);
        - cobertura mínima: autenticação/autorização, validações, regra de estoque/pedido, pagamento mock negado.
    - Critérios de avaliação:
        - Insuficiente (0–2): não há cenários claros ou não há evidência executável.
        - Básico (3–5): testes existem, mas não cobrem negativos ou não são reproduzíveis.
        - Bom (6–8): cobertura adequada e evidência consistente.
        - Excelente (9–10): cobertura forte + organização + testes negativos bem pensados (e/ou automatizados).

- **10 PONTOS: Entrega Técnica e Documentação (10 pontos):** Avalia a qualidade do material entregue e facilidade de validação pelo corretor.
    - Serão considerados:
        - PDF bem estruturado (seções, clareza, referências quando usadas);

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

- README completo e **reprodutível** (rodar, migrations, seed, env);
- links públicos e funcionais: repositório + Swagger + coleção Postman/Insomnia;
- organização do repositório (pastas, nomes, .env.example).

- **Critérios de avaliação:**
  - Insuficiente (0–2): links quebrados ou não é possível executar/testar.
  - Básico (3–5): documentação parcial, exige suposições do corretor.
  - Bom (6–8): documentação clara e execução reproduzível.
  - Excelente (9–10): entrega “profissional”: organizada, objetiva, sem lacunas.

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

Critérios de Avaliação (Back-end) — Tabela Consolidada (Critérios Oficiais)

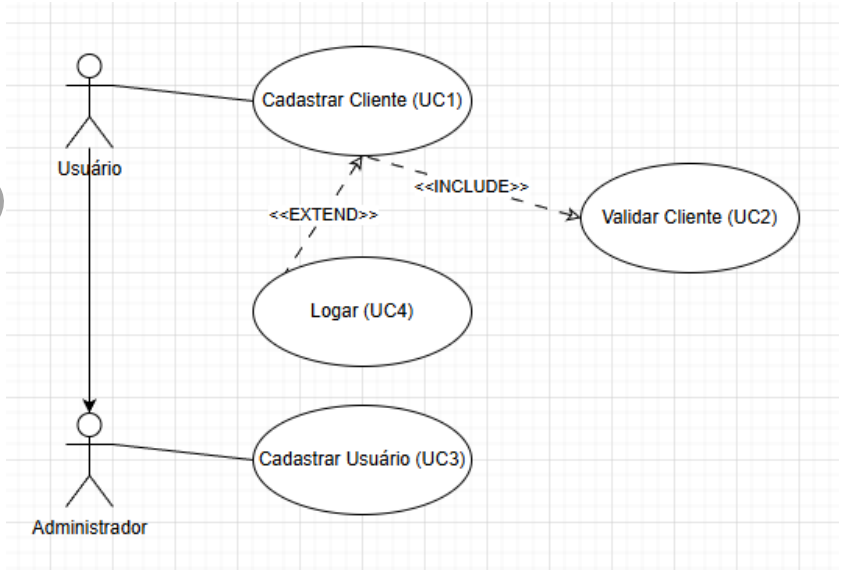
Observação: esta tabela consolida os critérios oficiais e a distribuição de pontos desta Atividade Prática. Em caso de divergência de interpretação entre texto e tabela, considere a Tabela Consolidada como referência principal.

Critério (pontos)	Evidências essenciais	Insuficiente	Básico	Bom	Excelente
1) Análise do Problema e Requisitos 20 pontos	• Contexto (rede/franquia; matriz e unidades) • RF/RNF de Back-end (API, dados, integrações, segurança) • Fidelização + pagamento mock + LGPD (mínimo) • Multicanalidade (canalPedido + rastreabilidade) • Priorização (implementado vs proposto) com justificativa	0–8: desconectado do caso; sem priorização	9–12: correto, porém genérico/pouco justificado	13–16: claro, coerente, priorizado	17–20: visão sistêmica + trade-offs + multicanalidade/LGPD/mock explícitos
2) Modelagem e Arquitetura 20 pontos	• Casos de uso • DER (PK/FK, cardinalidade, restrições) • Classes (domínio) e/ou sequência/atividade do fluxo crítico • Arquitetura por camadas (Domain/Application/Infrastructure/API) ou equivalente • Coerência modelagem ↔ regras ↔ endpoints	0–3: ausente/incoerente	4–8: correto, mas raso/pouco conectado	9–14: consistente e conectado ao fluxo crítico	15–20: maduro, bem justificado e alinhado ao que foi entregue
3) Implementação da API e Regras 25 pontos	• API executável (README) • Swagger/OpenAPI refletindo endpoints reais • Persistência (ORM/migrations) coerente com DER • Fluxo crítico fechado (pedido→pagamento mock→status OU estoque→movimentação→saldo) • Uso do canalPedido (criação/consulta) + validações + erro padrão • Auth (ex.: JWT) + 1 regra de autorização (roles)	0–5: não executa / não fecha mínimo / sem persistência	6–10: parcial; pouca coerência e/ou documentação fraca	11–17: fluxo concluído; persistência ok; endpoints coerentes	18–25: solução madura (erros, validações, status codes, paginação quando aplicável)
4) Segurança, LGPD, Logs/Auditoria 15 pontos	• Autenticação e autorização aplicadas • LGPD mínimo: finalidade, minimização e cuidado com dados • Armazenamento seguro (hash de senha; não expor dados sensíveis) • Logs/auditoria de ações sensíveis (ex.: criação/cancelamento, mudança de status)	0–3: ausente ou só citação	4–7: controles mínimos incompletos	8–11: coerente (auth/roles + LGPD mínimo + logs básicos)	12–15: integrado e consistente (rastreabilidade + justificativas)
5) Plano de Testes 10 pontos	• ≥10 cenários (mín. 6 positivos, 4 negativos) com entrada/saída esperada • Evidência executável: coleção Postman/Insomnia (preferencial) • Cobertura mínima: 401/403, 400/422, regra negócio (ex.: 409), pagamento mock negado • Referência da evidência (nome da requisição na coleção e/ou print complementar)	0–2: sem cenários claros ou sem evidência executável	3–5: existe, mas pouca cobertura/sem reprodutibilidade	6–8: cobertura adequada + evidência consistente	9–10: excelente organização/cobertura e/ou testes automatizados bem descritos
6) Entrega Técnica e Documentação 10 pontos	• PDF organizado (seções do roteiro) • README reprodutível (.env.example, migrations/seed, como rodar) • Links públicos e funcionais: repositório + Swagger + coleção • Organização do repositório (pastas, nomes, arquivos)	0–2: links quebrados ou impossível validar	3–5: documentação parcial; exige suposições	6–8: claro e reproduzível	9–10: padrão profissional; validação rápida pelo tutor

ANEXOS

EXEMPLO DE CASOS DE USO

Diagrama



EXEMPLO

Descrição de Casos de Usos (Modelo)

UC 1 – Cadastrar Cliente

Descrição

Cadastro de clientes que serão acessados por outros módulos do sistema.

Ator principal

Usuário do sistema *web*.

Demais atores

N/C

Pré-condições

1. Usuário do sistema web já deve ter sido cadastrado previamente.

Pós-condições

1. O sistema emite aviso.

Origem das informações

1. Base de Dados da Empresa Super Tratores;
2. Usuários do sistema *web*;
3. Usuários do sistema *mobile*;

EXEMPLO

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

Usuários responsáveis

N/C

Pendências

N/C

Fluxo base

1. Sistema apresenta a Tela de Cadastro de Clientes;
2. Usuário do sistema digita o *NOME DO CLIENTE;
3. Usuário do sistema digita o número do *TELEFONE 1 do cliente; [RN 1]
4. Usuário do sistema digita o número do TELEFONE 2 do cliente;
5. Usuário do sistema seleciona o VENDEDOR;
6. Usuário do sistema clica em *CADASTRAR;
7. Sistema emite aviso;

Fluxo alternativo

N/C

Regras de negócio

RN 1. O número do *TELEFONE 1 é o ID do cliente.

EXEMPLO

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

Modelo (template) para documentar endpoints

Use o modelo abaixo para cada endpoint:

[Recurso] — [Nome do endpoint]

Finalidade: …
Método/Rota: GET /recurso/{id}
Auth/Permissão: ( ) público ( ) JWT | Perfis: …
Parâmetros:
Path: {id} (tipo) — descrição
Query: page, limit, sort — descrição

Request (JSON):

```
{ }
```

Response 200/201 (JSON):

```
{ }
```

Status codes: 200, 400, 401, 403, 404, 409, 422
Erros (padrão):

```
    {
  "error": "NOME_DO_ERRO",
  "message": "Mensagem legível",
  "details": [
    {"field": "campo", "issue": "problema"}
  ],
  "timestamp": "2026-02-05T12:00:00Z",
  "path": "/rota",
  "requestId": "uuid-opcional"
}
```

EXEMPLO

Dica: se você estiver usando Swagger/OpenAPI, este “template” pode ser substituído **por prints do Swagger** + **exemplos de request/response** (mas o padrão de erro continua obrigatório).

Exemplos:

/auth — Login

Finalidade: autenticar usuário e retornar token.
Método/Rota: POST /auth/login
Auth/Permissão: público

Request:

```
{
  "email": "cliente@exemplo.com",
  "senha": "Senha@123"
}
```

EXEMPLO

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

Response 200:

```
{
  "accessToken": "jwt...",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "user": {
    "id": 10,
    "nome": "Maria",
    "perfil": "CLIENTE"
  }
}
```

Status codes: 200, 400, 401, 422
Erro 401 (exemplo):

```
{
  "error": "CREDENCIAIS_INVALIDAS",
  "message": "E-mail ou senha inválidos.",
  "details": [],
  "timestamp": "2026-02-05T12:00:00Z",
  "path": "/auth/login"
}
```

EXEMPLO

/pedidos — Criar pedido (fluxo crítico)

Finalidade: criar um pedido validando estoque e calculando total.
Método/Rota: POST /pedidos
Auth/Permissão: JWT | Perfis: CLIENTE

Request:

```
{
  "unidadeId": 2,
  "itens": [
    {"produtoId": 101, "quantidade": 2},
    {"produtoId": 305, "quantidade": 1}
  ],
  "formaPagamento": "PIX"
}
```

Response 201:

```
{
  "pedidoId": 9001,
  "status": "AGUARDANDO_PAGAMENTO",
  "total": 129.70,
  "itens": [
    {"produtoId": 101, "quantidade": 2, "precoUnitario": 39.90},
    {"produtoId": 305, "quantidade": 1, "precoUnitario": 49.90}
  ],
  "createdAt": "2026-02-05T12:00:00Z"
}
```

EXEMPLO

VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE

Status codes:

- 201 criado
- 401/403 sem permissão
- 404 unidade/produto inexistente
- 409 sem estoque (conflito)
- 422 request inválido

EXEMPLO

Erro 409 (sem estoque):

```
{
  "error": "ESTOQUE_INSUFICIENTE",
  "message": "Não há quantidade suficiente para um ou mais itens.",
  "details": [
    {"field": "itens[0].quantidade", "issue": "Disponível: 1"}
  ],
  "timestamp": "2026-02-05T12:00:00Z",
  "path": "/pedidos"
}
```


VERSAO EDITADA PARA ESTUDO | HA CONTEUDO OMITIDO | O ORIGINAL PREVALECE